UNIT II Relational Database Design

12 Codd's Rules

Every database has tables, and constraints cannot be referred to as a rational database system. And if any database has only relational data model, it cannot be a **Relational Database System (RDBMS)**. So, some rules define a database to be the correct RDBMS. These rules were developed by **Dr. Edgar F. Codd (E.F. Codd)** in **1985**, who has vast research knowledge on the Relational Model of database Systems. Codd presents his 13 rules for a database to test the concept of DBMS against his relational model, and if a database follows the rule, it is called a **true relational database (RDBMS)**. These 13 rules are popular in RDBMS, known as **Codd's 12 rules**.

Rule 0: The Foundation Rule

The database must be in relational form. So that the system can handle the database through its relational capabilities.

Rule 1: Information Rule

A database contains various information, and this information must be stored in each cell of a table in the form of rows and columns.

Rule 2: Guaranteed Access Rule

Every single or precise data (atomic value) may be accessed logically from a relational database using the combination of primary key value, table name, and column name.

Rule 3: Systematic Treatment of Null Values

This rule defines the systematic treatment of Null values in database records. The null value has various meanings in the database, like missing the data, no value in a cell, inappropriate information, unknown data and the primary key should not be null.

Rule 4: Active/Dynamic Online Catalog based on the relational model

It represents the entire logical structure of the descriptive database that must be stored online and is known as a database dictionary. It authorizes users to access the database and implement a similar query language to access the database.

Rule 5: Comprehensive Data SubLanguage Rule

The relational database supports various languages, and if we want to access the database, the language must be the explicit, linear or well-defined syntax, character strings and supports the comprehensive: data definition, view definition, data manipulation, integrity constraints, and limit transaction management operations. If the database allows access to the data without any language, it is considered a violation of the database.

Rule 6: View Updating Rule

All views table can be theoretically updated and must be practically updated by the database systems.

Rule 7: Relational Level Operation (High-Level Insert, Update and delete) Rule

A database system should follow high-level relational operations such as insert, update, and delete in each level or a single row. It also supports union, intersection and minus operation in the database system.

Rule 8: Physical Data Independence Rule

All stored data in a database or an application must be physically independent to access the database. Each data should not depend on other data or an application. If data is updated or the physical structure of the database is changed, it will not show any effect on external applications that are accessing the data from the database.

Rule 9: Logical Data Independence Rule

It is similar to physical data independence. It means, if any changes occurred to the logical level (table structures), it should not affect the user's view (application). For example, suppose a table either split into two tables, or two table joins to create a single table, these changes should not be impacted on the user view application.

Rule 10: Integrity Independence Rule

A database must maintain integrity independence when inserting data into table's cells using the SQL query language. All entered values should not be changed or rely on any external factor or application to maintain integrity. It is also helpful in making the database-independent for each front-end application.

Rule 11: Distribution Independence Rule

The distribution independence rule represents a database that must work properly, even if it is stored in different locations and used by different end-users. Suppose a user accesses the database through an application; in that case, they should not be aware that another user uses particular data, and the data they always get is only located on one site. The end users can access the database, and these access data should be independent for every user to perform the SQL queries.

Rule 12: Non Subversion Rule

The non-submersion rule defines RDBMS as a SQL language to store and manipulate the data in the database. If a system has a low-level or separate language other than SQL to access the database system, it should not subvert or bypass integrity to transform data.

Relational Integrity

What are Integrity Constraints ?

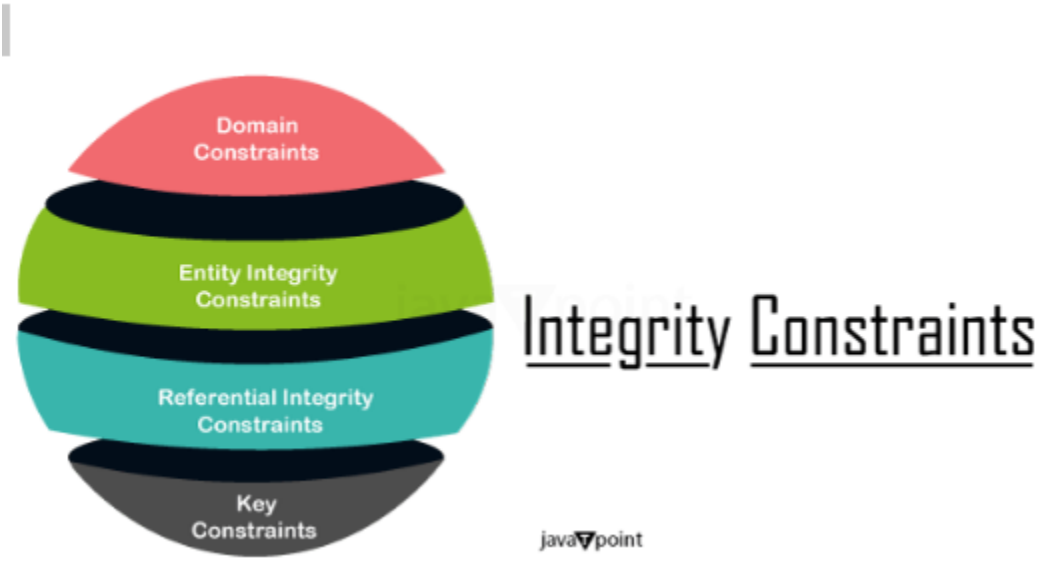
Integrity constraints in a Database Management System (DBMS) are rules that help keep the data in a database accurate, consistent and reliable. They act like a set of guidelines that ensure all the information stored in the database follows specific standards.
For example:

- Making sure every customer has a valid email address.
- Ensuring that an order in the database is always linked to an existing customer.

Integrity Constraints

- Integrity constraints are a set of rules. It is used to maintain the quality of information.
- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- Thus, integrity constraint is used to guard against accidental damage to the database.

Types of Integrity Constraint



1. Domain constraints

Domain constraints are a type of integrity constraint that ensure the values stored in a column (or attribute) of a database are valid and within a specific range or domain. In simple terms, they define what type of data is allowed in a column and restrict invalid data entry. The data type of domain include string, char, time, integer, date, currency etc. The value of the attribute must be available in comparable domains.

- Domain constraints can be defined as the definition of a valid set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1004	Morgan	8th	A

Not allowed. Because AGE is an integer attribute

2. Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

Example:

EMPLOYEE

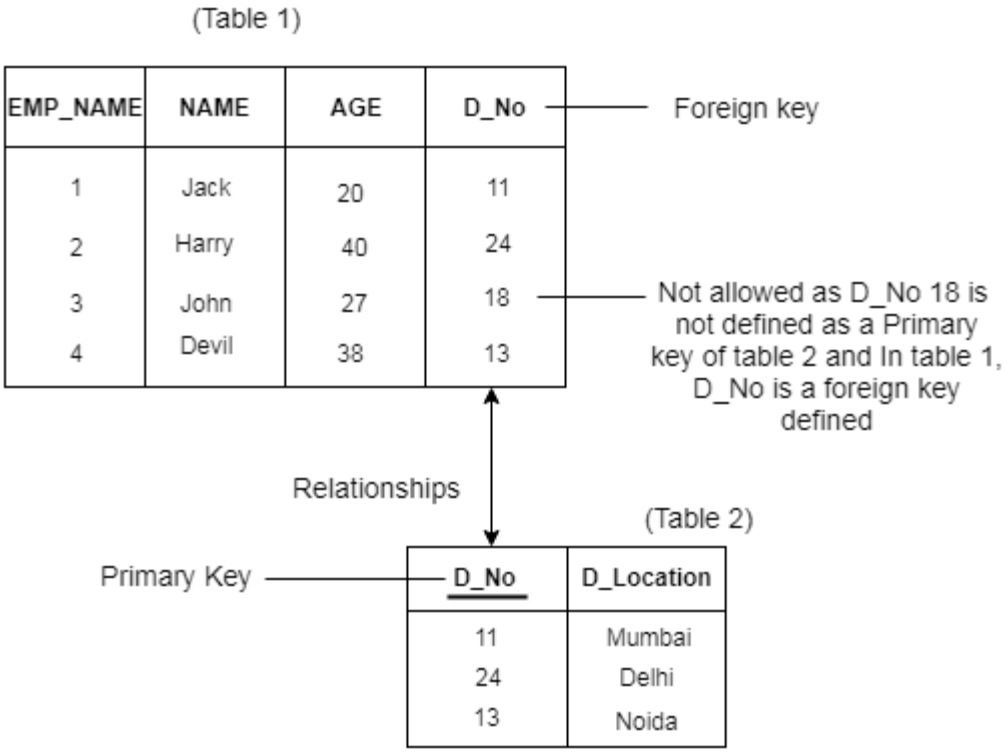
EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Example:



4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1002	Morgan	8th	22

Not allowed. Because all row must be unique

.

Primary Key Constraints

It states that the primary key attributes are required to be unique and not null. That is, primary key attributes of a relation must not have null values and primary key attributes of two tuples must never be same. This constraint is specified on database schema to the primary key attributes to ensure that no two tuples are same.

Example

Here, in the below example the Student_id is the primary key attribute. The data entry of 4th tuple violates the primary key constraint that is specifies on the database schema and therefore this instance of database is not a legal instance.

Student_id	Name	Semester	Age
101	Ramesh	5th	20
102	Kamlesh	5th	21
103	Akash	5th	22

Unique Values: Each `student_id` must be unique.

- `101`, `102`, `103` are valid.
- Inserting `101` again would result in an error.

Not NULL: `student_id` cannot be `NULL`.

- Valid: `103`.
- Invalid: A row with `NULL` for `student_id` will be rejected.

Features of Good Relational Designs:

- **Normalization:**

  Breaking down complex data into smaller, well-defined tables with minimal redundancy by applying normalization rules (1NF, 2NF, 3NF, etc.).

- **Data Integrity:**

  Ensuring accuracy and consistency of data through constraints like primary keys, foreign keys, and data validation rules.

- **Minimal Redundancy:**

  Avoiding unnecessary repetition of data across tables to maintain data consistency and reduce storage space.

- **Entity-Relationship Model (ERM):**

Clearly defining entities and their relationships within the database using an ER diagram.

- **Foreign Keys:**

  Using foreign keys to establish relationships between tables and enforce referential integrity.

- **No Modification Anomalies:**

  Preventing issues like update, insert, and delete anomalies where updating data in one table could unintentionally affect data in another.

- **Well-Defined Columns:**

  Each column should represent a single attribute with a clear data type.

- **Minimal Null Values:**

  Limiting the use of null values to situations where data is genuinely unknown to avoid data inconsistencies.

- **Proper Joins:**
  Designing tables and relationships to enable efficient and meaningful joins without generating incorrect results.

Normalization:-

- Database normalization is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).
- Data redundancy unnecessarily increases the size of the database as the same data is repeated in many places. Inconsistency problems also arise during insert, delete, and update operations.
- In the relational model, there exist standard methods to quantify how efficient a databases is. These methods are called normal forms and there are algorithms to covert a given database into normal forms.
- Normalization generally involves splitting a table into multiple ones which must be linked each time a query is made requiring data from the split tables.

Why do we need Normalization?

The primary objective for normalizing the relations is to eliminate the below anomalies. Failure to reduce anomalies results in data redundancy, which may threaten data integrity and cause additional issues as the database increases. Normalization consists of a set of procedures that assist you in developing an effective database structure.

- **Insertion Anomalies:** Insertion anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. For example, if a database requires that every record has a primary key, but no value is provided for a particular record, it cannot be inserted into the database.

- **Deletion anomalies:** Deletion anomalies occur when deleting a record from a database and can result in the unintentional loss of data. For example, if a database contains information about customers and orders, deleting a customer record may also delete all the orders associated with that customer.
- **Updation anomalies:** Updation anomalies occur when modifying data in a database and can result in inconsistencies or errors. For example, if a database contains information about employees and their salaries, updating an employee's salary in one record but not in all related records could lead to incorrect calculations and reporting.

Features of Database Normalization

- **Elimination of Data Redundancy:** One of the main features of normalization is to eliminate the data redundancy that can occur in a database. Data redundancy refers to the repetition of data in different parts of the database. Normalization helps in reducing or eliminating this redundancy, which can improve the efficiency and consistency of the database.
- **Ensuring Data Consistency:** Normalization helps in ensuring that the data in the database is consistent and accurate. By eliminating redundancy, normalization helps in preventing inconsistencies and contradictions that can arise due to different versions of the same data.
- **Simplification of Data Management:** Normalization simplifies the process of managing data in a database. By breaking down a complex data structure into simpler tables, normalization makes it easier to manage the data, update it, and retrieve it.
- **Improved Database Design:** Normalization helps in improving the overall design of the database. By organizing the data in a structured and systematic way, normalization makes it easier to design and maintain the database. It also makes the database more flexible and adaptable to changing business needs.
- **Avoiding Update Anomalies:** Normalization helps in avoiding update anomalies, which can occur when updating a single record in a table affects multiple records in other tables. Normalization ensures that each table contains only one type of data and that the relationships between the tables are clearly defined, which helps in avoiding such anomalies.
- **Standardization:** Normalization helps in standardizing the data in the database. By organizing the data into tables and defining relationships between them, normalization helps in ensuring that the data is stored in a consistent and uniform manner.

First Normal Form (1NF)

If a relation contains a composite or multi-valued attribute, it violates the first normal form, or the relation is in the first normal form if it does not contain any composite or multi-valued attribute. A relation is in first normal form if every attribute in that relation is single-valued attribute.

A table is in 1 NF if:

- There are only Single Valued Attributes.
- Attribute Domain does not change.
- There is a unique name for every Attribute/Column.
- The order in which data is stored does not matter.

Rules for First Normal Form (1NF) in DBMS

To follow the First Normal Form (1NF) in a database, these simple rules must be followed:

1. Every Column Should Have Single Values

Each column in a table must contain only one value in a cell. No cell should hold multiple values. If a cell contains more than one value, the table does not follow 1NF.

- **Example:** A table with columns like [Writer 1], [Writer 2], and [Writer 3] for the same book ID is not in 1NF because it repeats the same type of information (writers). Instead, all writers should be listed in separate rows.

2. All Values in a Column Should Be of the Same Type

Each column must store the same type of data. You cannot mix different types of information in the same column.

- **Example:** If a column is meant for dates of birth (DOB), you cannot use it to store names. Each type of information should have its own column.

3. Every Column Must Have a Unique Name

Each column in the table must have a unique name. This avoids confusion when retrieving, updating, or adding data.

- **Example:** If two columns have the same name, the database system may not know which one to use.

4. The Order of Data Doesn't Matter

In 1NF, the order in which data is stored in a table doesn't affect how the table works. You can organize the rows in any way without breaking the rules.

Example:

Consider the below COURSES Relation :

COURSES Table

ID	Name	Courses
1	A	c1, c2
2	E	c3
3	M	c2, c3

- In the above table, Courses has a multi-valued attribute, so it is not in 1NF. The Below Table is in 1NF as there is no multi-valued attribute.

COURSES Table

ID	Name	Courses
1	A	c1
1	A	c2
2	E	c3
3	M	c2
3	M	c3

Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key

Example: Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

TEACHER table

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Computer	38

In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

TEACHER_DETAIL table:

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

TEACHER_SUBJECT table:

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math

Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency X → Y.

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

To explain the 3NF, let us consider the example of Employee_Detail relation as shown below.

Example: EMPLOYEE_DETAIL table

EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Super key in the table above:

1. {EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}....so on

   Candidate key: {EMP_ID}

   Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

   Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). It violates the rule of third normal form.The reduction of 2NF relation into 3NF consists of splitting the 2NF into appropriate relations such that every non-key attribute are functionally dependent on the primary key not transitively or indirectly of the respective relations.

   That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

   EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich

Boyce Codd normal form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency X → Y, X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

Example: Let's assume there is a company where employees work in more than one department.

EMPLOYEE table:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

In the above table Functional dependencies are as follows:

1. EMP_ID → EMP_COUNTRY
2. EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate key: {EMP-ID, EMP-DEPT}
The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.

To convert the given table into BCNF, we decompose it into three tables:

EMP_COUNTRY table:

EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India

EMP_DEPT table:

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING table:

EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

Functional dependencies:

> EMP_ID → EMP_COUNTRY
> EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate keys:

For the first table: EMP_ID
For the second table: EMP_DEPT
For the third table: {EMP_ID, EMP_DEPT}

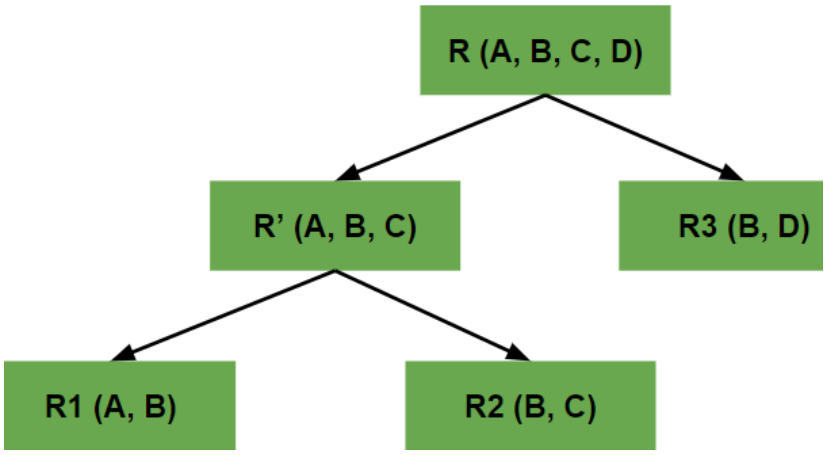
Now, this is in BCNF because left side part of both the functional dependencies is a key.

Decomposition using functional dependency

Decomposition refers to the division of tables into multiple tables to produce consistency in the data. In this article, we will learn about the Database concept. This article is related to the concept of Decomposition in DBMS. It explains the definition of Decomposition, types of Decomposition in DBMS, and its properties.

What is Decomposition in DBMS?

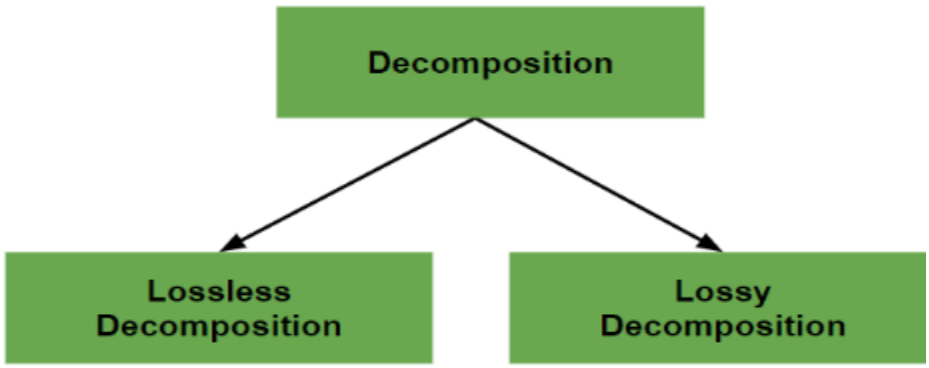
When we divide a table into multiple tables or divide a relation into multiple relations, then this process is termed Decomposition in DBMS. We perform decomposition in DBMS when we want to process a particular data set. It is performed in a database management system when we need to ensure consistency and remove anomalies and duplicate data present in the database. When we perform decomposition in DBMS, we must try to ensure that no information or data is lost.



Types of Decomposition

There are two types of Decomposition:

- Lossless Decomposition
- Lossy Decomposition



Lossless Decomposition

The process in which where we can regain the original relation R with the help of joins from the multiple relations formed after decomposition. This process is termed as lossless decomposition. It is used to remove the redundant data from the database while retaining the useful information. The lossless decomposition tries to ensure following things:

- While regaining the original relation, no information should be lost.
- If we perform join operation on the sub-divided relations, we must get the original relation.

Example:
There is a relation called R(A, B, C)

A	B	C
55	16	27
48	52	89

Now we decompose this relation into two sub relations R1 and R2
R1(A, B)

A	B
55	16
48	52

R2(B, C)

B	C
16	27
52	89

After performing the Join operation we get the same original relation

A	B	C
55	16	27
48	52	89

Lossy Decomposition

As the name suggests, lossy decomposition means when we perform join operation on the sub-relations it doesn't result to the same relation which was decomposed. After the join operation, we always found some extraneous tuples. These extra tuples genrates difficulty for the user to identify the original tuples.
Example:
We have a relation R(A, B, C)

A	B	C
1	2	1
2	5	3
3	3	3

Now , we decompose it into sub-relations R1 and R2
R1(A, B)

A	B
1	2
2	5
3	3

R2(B, C)

B	C
2	1
5	3
3	3

Now After performing join operation

A	B	C
1	2	1
2	5	3
2	3	3
3	5	3
3	3	3